SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – DESIGN TASK

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO3 – Precision	**Assessment:**	Assessment Tool 3 – ERROR ANALYSIS
Weightage:	30% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	N. Tharun Sai	**Reg. No.:**	192472382
Section / Batch:	CSE - AI	**Date:**	20/8/2026

Code of Conduct

I N. Tharun Sai (192472382) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: N.Tharun Sai

Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Inflectional Morphology and Derivational Morphology	**Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.**	**Q1**
Inflectional Morphology and Derivational Morphology	**Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.**	**Q2**
Finite-State Morphological Parsing	**Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.**	**Q3**
Finite-State Morphological Parsing	**Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.**	**Q4**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a Porter Stemmer implementation for words such as *connected, connection, connecting,* and *connectivity*; identify stemming-rule errors and correct the implementation.**	**Q5**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a morphological normalization program that processes *studies, studied, studying,* and *study*; identify incorrect stemming outputs and modify the code to produce consistent stems.**	**Q6**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.**	**Q7**

Annexure A – Design Task

Task 1 – N-gram Based Text Prediction

Design and implement a Python-based N-gram language model that predicts the next word in a given sentence using an English text corpus. The system should preprocess the corpus, perform sentence segmentation and tokenization, and construct unigram, bigram, and trigram models by calculating word-frequency counts and maximum-likelihood probabilities.

For an incomplete sentence such as:

"Artificial intelligence is"

the system should identify and display the top-5 most probable next words.

The program should allow the user to select N = 1, 2, or 3, display the corresponding N-gram frequency counts and probabilities, and demonstrate the behavior of the model when an unseen N-gram is encountered. Finally, test the model using multiple incomplete sentences, calculate the prediction accuracy, and discuss the limitations of using an unsmoothed N-gram model for real-world language prediction.

Task 2 – Backoff and Interpolation for Language Prediction

Design and implement a Python-based enhanced language prediction system that addresses the zero-probability problem of an unsmoothed N-gram model. The system should construct unigram, bigram, and trigram models from an English corpus and accept an incomplete sentence such as:

"Machine learning can"

When the required trigram is unavailable, the system should use a backoff strategy by first checking the trigram model, then the bigram model, and finally the unigram model. Extend the system using Deleted Interpolation to combine unigram, bigram, and trigram probabilities using predefined interpolation weights. The system should generate the top-5 next-word predictions using:

- Unsmoothed N-gram model
- Backoff model
- Deleted Interpolation model

Compare the three approaches in terms of zero probabilities, prediction coverage, and prediction accuracy, and explain why backoff and interpolation are useful for sparse language data.

Task 3 – Entropy-Based Evaluation of Language Models

Design and implement a Python program for measuring the uncertainty of N-gram language models using entropy. Given separate training and testing corpora, construct unigram, bigram, and trigram language models and calculate the probabilities assigned to words in the test sentences. For each test sentence, calculate its entropy and classify the sentence as having relatively high or low predictive uncertainty.

For example, compare sentences such as:

"The cat sits on the mat." and

"The quantum processor redesigned the..."

The system should evaluate how predictable the next word is based on the trained language model. The program should also compare entropy values obtained using:

- Unigram model
- Bigram model
- Trigram model
- Smoothed model

Finally, interpret the results and explain how entropy can be used to measure prediction uncertainty and evaluate the reliability of a language model.

Task 4 – Comparative POS Tagging System

Design and implement a Python-based POS tagging system that assigns grammatical tags to words in English sentences.

The system should implement three approaches:

1. Rule-Based POS Tagging

Use a combination of lexical dictionaries, word patterns, and grammatical rules to assign POS tags.

2. Stochastic POS Tagging

Construct a statistical POS tagger using:

- Word/tag frequencies
- Tag transition probabilities
- Probabilities obtained from a training corpus

3. Transformation-Based Tagging

Initially assign tags using a simple tagging strategy and subsequently apply transformation rules to correct likely tagging errors.

For example:

"I book a ticket."

and

"I read a book."

The system should demonstrate how the same word can receive different POS tags depending on its context. Use an appropriate English corpus such as the Brown Corpus or another publicly available tagged corpus. The program should accept user-entered sentences, display the POS tags produced by all three approaches, and compare their results using suitable evaluation measures such as tagging accuracy. Finally, analyze the differences among the three approaches and explain the advantages and limitations of rule-based, stochastic, and transformation-based POS tagging.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Error Identification	Root Cause Analysis	Correction & Justification	Applications of Concepts	Clarity & Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1.N-Gram Based Text Prediction Using a Language Model

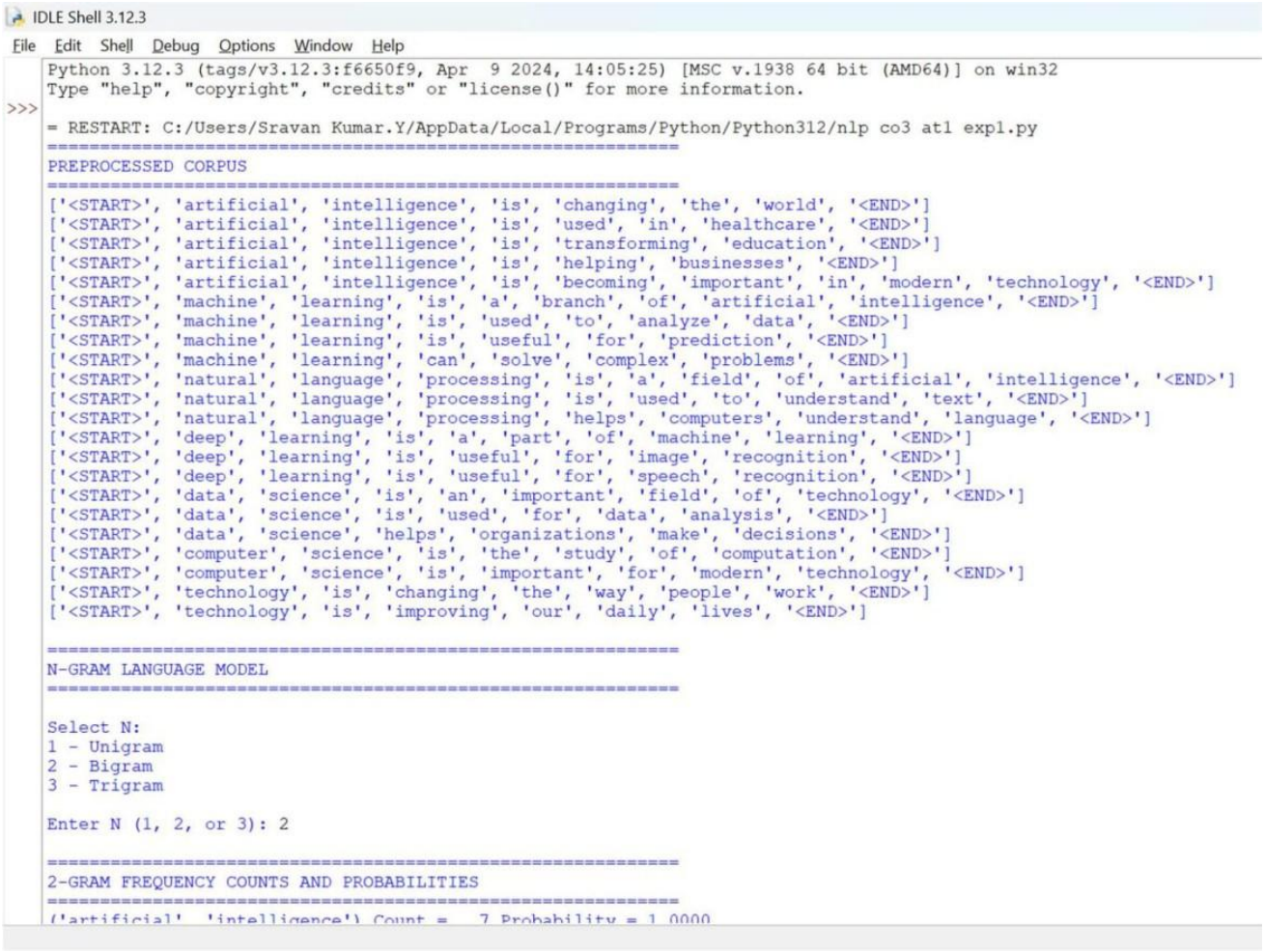
CODE

nlp co3 at1 exp1.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/nlp co3 at1 exp1.py (3.12.3)

File Edit Format Run Options Window Help

```
import re
from collections import Counter, defaultdict


# ============================================================
# 1. ENGLISH TEXT CORPUS
# ============================================================

corpus = """
Artificial intelligence is changing the world.
Artificial intelligence is used in healthcare.
Artificial intelligence is transforming education.
Artificial intelligence is helping businesses.
Artificial intelligence is becoming important in modern technology.

Machine learning is a branch of artificial intelligence.
Machine learning is used to analyze data.
Machine learning is useful for prediction.
Machine learning can solve complex problems.

Natural language processing is a field of artificial intelligence.
Natural language processing is used to understand text.
Natural language processing helps computers understand language.

Deep learning is a part of machine learning.
Deep learning is useful for image recognition.
Deep learning is useful for speech recognition.

Data science is an important field of technology.
Data science is used for data analysis.
Data science helps organizations make decisions.

Computer science is the study of computation.
Computer science is important for modern technology.
Technology is changing the way people work.
Technology is improving our daily lives.
"""


# ============================================================
# 2. SENTENCE SEGMENTATION
# ============================================================

def sentence_segmentation(text):
    sentences = re.split(r'[.!?]+', text)
```

OUTPUT



```
IDLE Shell 3.12.3
File  Edit  Shell  Debug  Options  Window  Help
Python 3.12.3 (tags/v3.12.3:f6650f9, Apr  9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/nlp co3 at1 exp1.py
============================================================
PREPROCESSED CORPUS
============================================================
['<START>', 'artificial', 'intelligence', 'is', 'changing', 'the', 'world', '<END>']
['<START>', 'artificial', 'intelligence', 'is', 'used', 'in', 'healthcare', '<END>']
['<START>', 'artificial', 'intelligence', 'is', 'transforming', 'education', '<END>']
['<START>', 'artificial', 'intelligence', 'is', 'helping', 'businesses', '<END>']
['<START>', 'artificial', 'intelligence', 'is', 'becoming', 'important', 'in', 'modern', 'technology', '<END>']
['<START>', 'machine', 'learning', 'is', 'a', 'branch', 'of', 'artificial', 'intelligence', '<END>']
['<START>', 'machine', 'learning', 'is', 'used', 'to', 'analyze', 'data', '<END>']
['<START>', 'machine', 'learning', 'is', 'useful', 'for', 'prediction', '<END>']
['<START>', 'machine', 'learning', 'can', 'solve', 'complex', 'problems', '<END>']
['<START>', 'natural', 'language', 'processing', 'is', 'a', 'field', 'of', 'artificial', 'intelligence', '<END>']
['<START>', 'natural', 'language', 'processing', 'is', 'used', 'to', 'understand', 'text', '<END>']
['<START>', 'natural', 'language', 'processing', 'helps', 'computers', 'understand', 'language', '<END>']
['<START>', 'deep', 'learning', 'is', 'a', 'part', 'of', 'machine', 'learning', '<END>']
['<START>', 'deep', 'learning', 'is', 'useful', 'for', 'image', 'recognition', '<END>']
['<START>', 'deep', 'learning', 'is', 'useful', 'for', 'speech', 'recognition', '<END>']
['<START>', 'data', 'science', 'is', 'an', 'important', 'field', 'of', 'technology', '<END>']
['<START>', 'data', 'science', 'is', 'used', 'for', 'data', 'analysis', '<END>']
['<START>', 'data', 'science', 'helps', 'organizations', 'make', 'decisions', '<END>']
['<START>', 'computer', 'science', 'is', 'the', 'study', 'of', 'computation', '<END>']
['<START>', 'computer', 'science', 'is', 'important', 'for', 'modern', 'technology', '<END>']
['<START>', 'technology', 'is', 'changing', 'the', 'way', 'people', 'work', '<END>']
['<START>', 'technology', 'is', 'improving', 'our', 'daily', 'lives', '<END>']

============================================================
N-GRAM LANGUAGE MODEL
============================================================

Select N:
1 - Unigram
2 - Bigram
3 - Trigram

Enter N (1, 2, or 3): 2

============================================================
2-GRAM FREQUENCY COUNTS AND PROBABILITIES
============================================================
('artificial', 'intelligence') Count =   7 Probability = 1.0000
```

CODE

```
import re
from collections import Counter, defaultdict


# ============================================================
# 1. ENGLISH TEXT CORPUS
# ============================================================

corpus = """
Artificial intelligence is changing the world.
Artificial intelligence is used in healthcare.
Artificial intelligence is transforming education.
Artificial intelligence is helping businesses.
Artificial intelligence is becoming important in modern technology.

Machine learning is a branch of artificial intelligence.
Machine learning is used to analyze data.
Machine learning is useful for prediction.
Machine learning can solve complex problems.

Natural language processing is a field of artificial intelligence.
Natural language processing is used to understand text.
```

Natural language processing helps computers understand language.

```
Deep learning is a part of machine learning.
Deep learning is useful for image recognition.
Deep learning is useful for speech recognition.

Data science is an important field of technology.
Data science is used for data analysis.
Data science helps organizations make decisions.

Computer science is the study of computation.
Computer science is important for modern technology.
Technology is changing the way people work.
Technology is improving our daily lives.
"""


# ================================================================
# 2. SENTENCE SEGMENTATION
# ================================================================

def sentence_segmentation(text):
    sentences = re.split(r'[.!?]+', text)

    sentences = [
        sentence.strip()
        for sentence in sentences
        if sentence.strip()
    ]

    return sentences


# ================================================================
# 3. TOKENIZATION
# ================================================================

def tokenize(sentence):
    return re.findall(r'\b[a-zA-Z]+\b', sentence.lower())


# ================================================================
# 4. PREPROCESS THE CORPUS
# ================================================================

sentences = sentence_segmentation(corpus)

tokenized_sentences = []

for sentence in sentences:
    tokens = tokenize(sentence)

    # Add sentence boundary tokens
    tokens = ["<START>"] + tokens + ["<END>"]
```

```
    tokenized_sentences.append(tokens)


print("=" * 60)
print("PREPROCESSED CORPUS")
print("=" * 60)

for sentence in tokenized_sentences:
    print(sentence)


# ============================================================
# 5. BUILD UNIGRAM, BIGRAM AND TRIGRAM COUNTS
# ============================================================

unigram_counts = Counter()
bigram_counts = Counter()
trigram_counts = Counter()

for tokens in tokenized_sentences:

    # Unigram
    for word in tokens:
        unigram_counts[word] += 1

    # Bigram
    for i in range(len(tokens) - 1):
        bigram = (tokens[i], tokens[i + 1])
        bigram_counts[bigram] += 1

    # Trigram
    for i in range(len(tokens) - 2):
        trigram = (tokens[i], tokens[i + 1], tokens[i + 2])
        trigram_counts[trigram] += 1


# ============================================================
# 6. MAXIMUM LIKELIHOOD PROBABILITIES
# ============================================================

# Unigram probability:
# P(word) = Count(word) / Total words

total_words = sum(unigram_counts.values())

unigram_probabilities = {}

for word, count in unigram_counts.items():
    unigram_probabilities[word] = count / total_words


# Bigram probability:
```

```
# P(word2 | word1) =
# Count(word1, word2) / Count(word1)

bigram_probabilities = {}

for (word1, word2), count in bigram_counts.items():
    bigram_probabilities[(word1, word2)] = (
        count / unigram_counts[word1]
    )


# Trigram probability:
# P(word3 | word1, word2) =
# Count(word1, word2, word3) / Count(word1, word2)

trigram_probabilities = {}

for (word1, word2, word3), count in trigram_counts.items():
    trigram_probabilities[(word1, word2, word3)] = (
        count / bigram_counts[(word1, word2)]
    )


# ================================================================
# 7. DISPLAY N-GRAM COUNTS AND PROBABILITIES
# ================================================================

def display_ngrams(n):

    print("\n" + "=" * 60)
    print(f"{n}-GRAM FREQUENCY COUNTS AND PROBABILITIES")
    print("=" * 60)

    if n == 1:

        for word, count in unigram_counts.most_common():
            probability = unigram_probabilities[word]

            print(
                f"{word:15s} Count = {count:3d} "
                f"Probability = {probability:.4f}"
            )

    elif n == 2:

        for bigram, count in bigram_counts.most_common():
            probability = bigram_probabilities[bigram]

            print(
                f"{bigram} Count = {count:3d} "
                f"Probability = {probability:.4f}"
            )
```

```
    elif n == 3:

        for trigram, count in trigram_counts.most_common():
            probability = trigram_probabilities[trigram]

            print(
                f"{trigram} Count = {count:3d} "
                f"Probability = {probability:.4f}"
            )


# ============================================================
# 8. PREDICT NEXT WORD
# ============================================================

def predict_next_word(sentence, n, top_k=5):

    tokens = tokenize(sentence)

    if len(tokens) == 0:
        return []

    predictions = []

    # ----------------------------------
    # UNIGRAM
    # ----------------------------------

    if n == 1:

        for word, probability in unigram_probabilities.items():

            if word not in ["<START>", "<END>"]:
                predictions.append((word, probability))

    # ----------------------------------
    # BIGRAM
    # ----------------------------------

    elif n == 2:

        last_word = tokens[-1]

        for (word1, word2), probability in bigram_probabilities.items():

            if word1 == last_word and word2 not in ["<START>", "<END>"]:
                predictions.append((word2, probability))

    # ----------------------------------
    # TRIGRAM
    # ----------------------------------

    elif n == 3:
```

```
        if len(tokens) < 2:
            return []

        word1 = tokens[-2]
        word2 = tokens[-1]

        for (w1, w2, w3), probability in trigram_probabilities.items():

            if w1 == word1 and w2 == word2:
                if w3 not in ["<START>", "<END>"]:
                    predictions.append((w3, probability))

    predictions.sort(key=lambda x: x[1], reverse=True)

    return predictions[:top_k]


# ============================================================
# 9. DISPLAY TOP-5 PREDICTIONS
# ============================================================

def display_prediction(sentence, n):

    print("\n" + "=" * 60)
    print("NEXT WORD PREDICTION")
    print("=" * 60)

    print("Sentence:", sentence)
    print("N =", n)

    predictions = predict_next_word(sentence, n)

    if not predictions:

        print("\nNo prediction available.")
        print("This is an unseen N-gram.")

        return

    print("\nTop-5 predicted next words:")

    for rank, (word, probability) in enumerate(predictions, 1):

        print(
            f"{rank}. {word:<15} "
            f"Probability = {probability:.4f}"
        )


# ============================================================
# 10. UNSEEN N-GRAM DEMONSTRATION
# ============================================================
```

```
def check_unseen_ngram(sentence, n):

    tokens = tokenize(sentence)

    print("\n" + "=" * 60)
    print("UNSEEN N-GRAM CHECK")
    print("=" * 60)

    if n == 2:

        if len(tokens) < 2:
            return

        bigram = (tokens[-2], tokens[-1])

        if bigram not in bigram_counts:

            print("Unseen Bigram:", bigram)
            print("Frequency = 0")
            print("Probability = 0")

        else:

            print("Bigram exists:", bigram)
            print(
                "Probability =",
                bigram_probabilities[bigram]
            )

    elif n == 3:

        if len(tokens) < 3:
            return

        trigram = (
            tokens[-3],
            tokens[-2],
            tokens[-1]
        )

        if trigram not in trigram_counts:

            print("Unseen Trigram:", trigram)
            print("Frequency = 0")
            print("Probability = 0")

        else:

            print("Trigram exists:", trigram)
            print(
                "Probability =",
                trigram_probabilities[trigram]
```

```
        )

# ================================================================
# 11. ACCURACY TEST
# ================================================================

test_cases = [

    ("Artificial intelligence is", "changing"),

    ("Machine learning is", "a"),

    ("Natural language processing is", "a"),

    ("Deep learning is", "a"),

    ("Data science is", "an"),

    ("Computer science is", "the"),

]


def calculate_accuracy(n):

    correct = 0
    total = len(test_cases)

    print("\n" + "=" * 60)
    print(f"ACCURACY TEST - N = {n}")
    print("=" * 60)

    for sentence, actual_word in test_cases:

        predictions = predict_next_word(
            sentence,
            n,
            top_k=5
        )

        predicted_words = [
            word for word, probability in predictions
        ]

        if actual_word in predicted_words:

            correct += 1
            result = "CORRECT"

        else:

            result = "WRONG"
```

```
        print(
            f"\nSentence: {sentence}"
        )

        print(
            f"Actual next word: {actual_word}"
        )

        print(
            f"Predictions: {predicted_words}"
        )

        print(
            f"Result: {result}"
        )

    accuracy = (correct / total) * 100

    print("\n" + "-" * 60)
    print(
        f"Accuracy = {correct}/{total} "
        f"= {accuracy:.2f}%"
    )
    print("-" * 60)


# ============================================================
# 12. MAIN PROGRAM
# ============================================================

print("\n" + "=" * 60)
print("N-GRAM LANGUAGE MODEL")
print("=" * 60)

print("\nSelect N:")
print("1 - Unigram")
print("2 - Bigram")
print("3 - Trigram")

n = int(input("\nEnter N (1, 2, or 3): "))

if n not in [1, 2, 3]:

    print("Invalid choice.")

else:

    # Display frequency counts and probabilities
    display_ngrams(n)

    # Input sentence
    sentence = input(
```

```
        "\nEnter an incomplete sentence: "
    )

    # Prediction
    display_prediction(sentence, n)

    # Unseen N-gram check
    if n in [2, 3]:
        check_unseen_ngram(sentence, n)

    # Accuracy
    calculate_accuracy(n)


# ============================================================
# 13. LIMITATIONS
# ============================================================

print("\n" + "=" * 60)
print("LIMITATIONS OF UNSMOOTHED N-GRAM MODEL")
print("=" * 60)

print("""
1. Unseen N-grams get probability 0.

2.The  model cannot predict word combinations that
   were not present in the training corpus.

3. It requires a large corpus for better predictions.

4. Unigram ignores word context.

5. Bigram and trigram models consider only limited context.

6.The  model cannot understand the actual meaning of
   words or sentences.

7. The model may perform poorly on new or rare sentences.
""")
```